IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Stavroula Katsioli
28th February 2026

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
  - Data Collection from an API
  - Data Collection with Web Scraping
  - Data Wrangling
  - Exploratory Data Analysis with SQL
  - Exploratory Data Analysis with Visualization
  - Interactive Visual Analytics with Folium
  - Machine Learning Prediction
- Summary of all results
  - Exploratory Data Analysis Results
  - Interactive Analytics in Screenshots
  - Predictive Analysis Results

Introduction

The commercial space industry has become increasingly competitive, with SpaceX leading the way in innovation and cost reduction. A major factor behind its success is the reusability of the Falcon 9 first stage, which significantly lowers launch costs compared to traditional providers. However, first-stage landings are not always successful and depend on mission conditions.

This project analyzes SpaceX launch data to predict whether the Falcon 9 first stage will be reused. Using publicly available data, visualizations, and machine learning, the goal is to build a predictive model that estimates landing outcomes and better understands launch cost efficiency.

- Main questions:
  - Which factors influence whether the rocket's first stage lands successfully?
  - How do different features relate to one another in affecting the likelihood of a successful landing?
  - What operational conditions are required to maximize the chances of a successful landing program?

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - Usage of SpaceX API
  - Web Scraping from Wikipedia
- Perform data wrangling
  - One-hot Encoding for Categorical Features
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
  - Logistic Regression, K Nearest Neighbor, Support Vector Machine, and Decision Tree models building, tuning and evaluating

Data Collection

- Data collection began with sending a GET request to the SpaceX API to retrieve launch data. The response was decoded into JSON format using the `.json()` method and then transformed into a structured pandas DataFrame with `pd.json_normalize()`.
- The dataset was subsequently cleaned by checking for missing values and imputing them where necessary to ensure consistency.
- Additionally, web scraping was performed on Wikipedia to gather Falcon 9 launch records using `BeautifulSoup`. The relevant HTML tables were extracted, parsed, and converted into a pandas DataFrame to support further analysis.

Data Collection – SpaceX API

- A GET request is sent using `requests.get()` to retrieve historical launch data.
- The JSON response is converted into a structured pandas DataFrame using `pd.json_normalize()`.
- Custom functions are called to extract and organize relevant features.
- Missing values are replaced with the mean value.
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Data%20collection%20API.ipynb

```
spacex_url="https://api.spacexdata.com/v4/launches/past"

response = requests.get(spacex_url)
```

```
data = pd.json_normalize(response)
```

```
getBoosterVersion(data)

getLaunchSite(data)

getPayloadData(data)

getCoreData(data)
```

```
payload_mean = data_falcon9['PayloadMass'].mean()

data_falcon9['PayloadMass'] =
data_falcon9['PayloadMass'].replace(np.nan,
payload_mean)
```

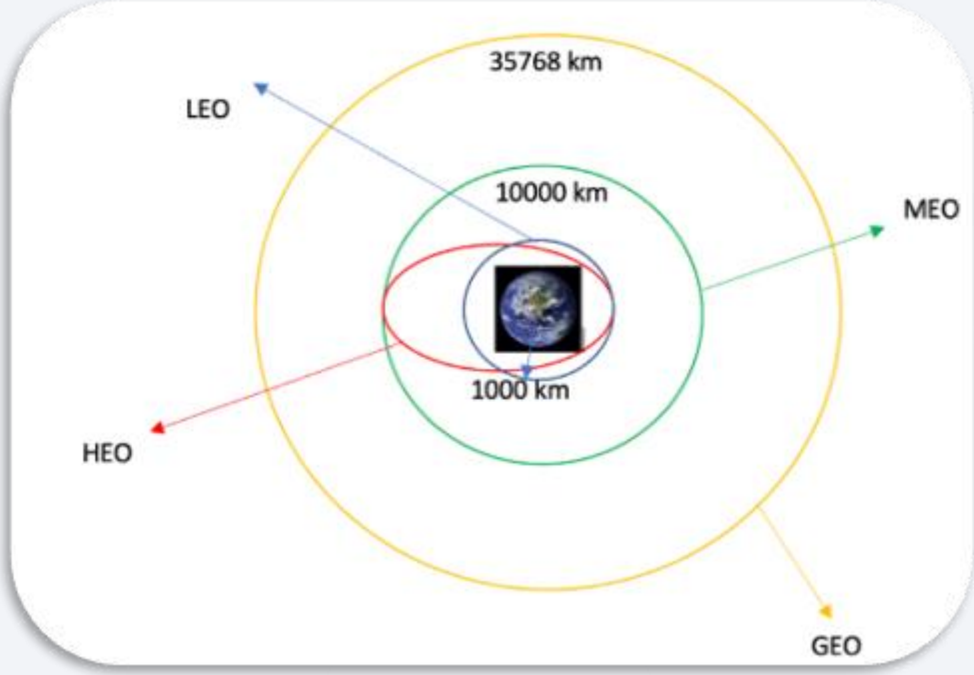
Data Collection - Scraping

- A GET request is sent to `static_url` to retrieve the webpage content.
- The HTML response is parsed using `BeautifulSoup` with the `html.parser`.
- All `<table>` elements are collected using `soup.find_all()`.
- Column names are extracted from table headers (`<th>`).
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Webscraping.ipynb

```
data = requests.get(static_url)
soup = BeautifulSoup(data.text, 'html.parser')
_________________________
html_tables = soup.find_all('table')
_________________________
column_names = []

for row in soup.find_all('th'):
    name = extract_column_from_header(row)
    if (name is not None and len(name) > 0):
        column_names.append(name)
launch_dict= dict.fromkeys(column_names)
_________________________
df= pd.DataFrame({ key:pd.Series(value) for key,
value in launch_dict.items() })
```

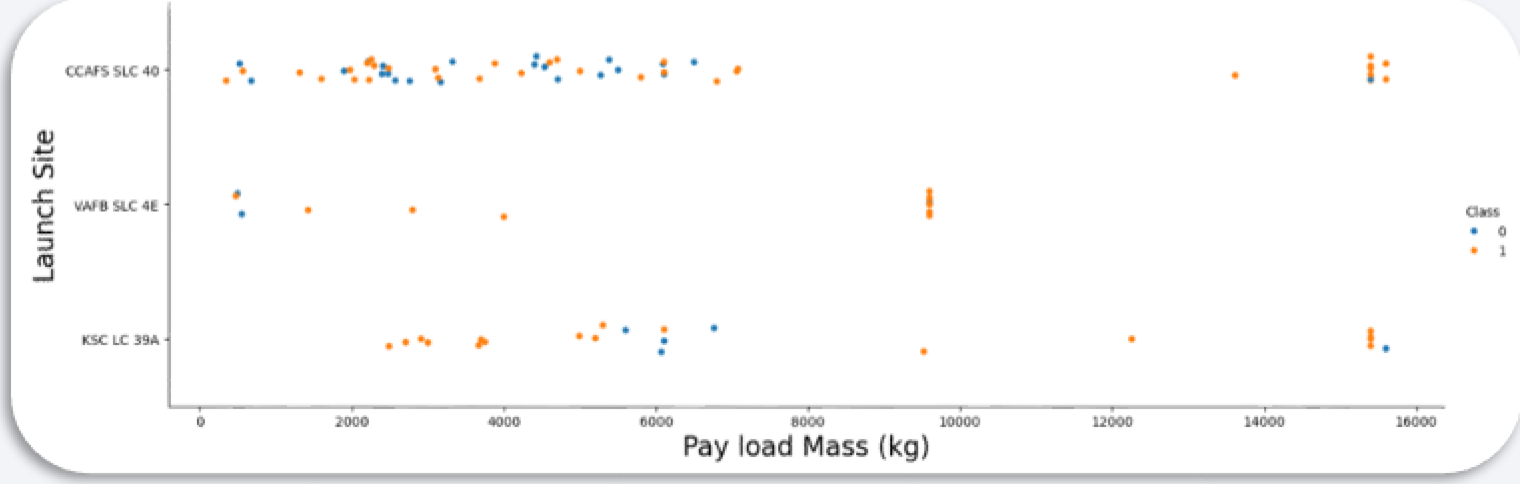
Data Wrangling

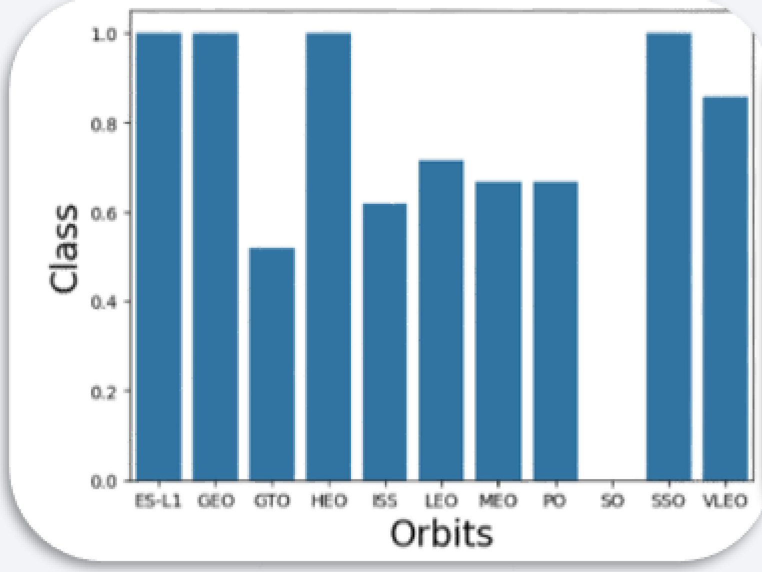
- Data wrangling involves cleaning, structuring, and consolidating complex datasets to make them suitable for analysis and exploratory data analysis (EDA).
- In this step, we first calculate the total number of launches at each site and analyze the frequency of mission outcomes by orbit type. We then create a binary landing outcome label based on the original outcome column to simplify further analysis, visualization, and machine learning tasks.
- Finally, the processed dataset is exported as a CSV file for subsequent use.
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Data%20wrangling.ipynb

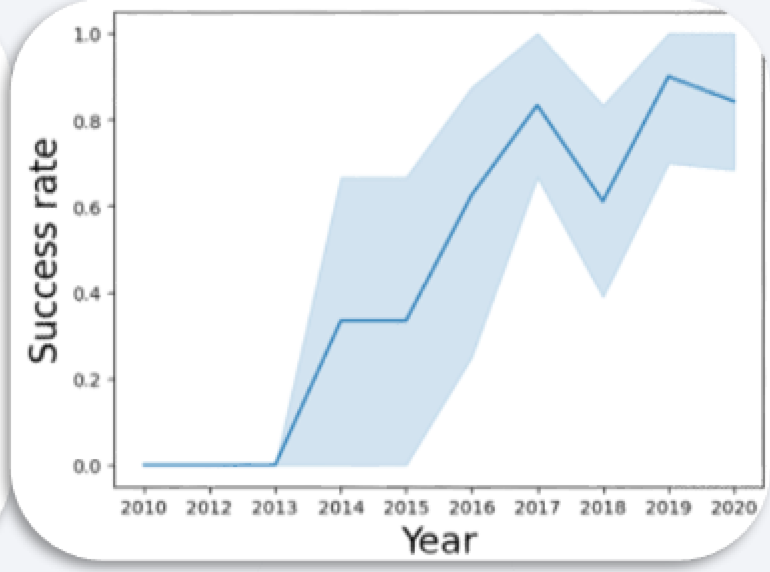


EDA with Data Visualization

- Using the `seaborn` pacakage, we explored the relationship between the different parameters though **scatterplots**, we groupped the results using **barplots**, and we explored the trends using **lineplots**.
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Exploratory%20Data%20Analytics%20Visualization.ipynb







EDA with SQL

- SQL queries were used to achieve a better understanding of the dataset:
  - Find the names of the unique launch sites in the space mission
  - Find specific records, e.g., where launch sites begin with specific letters
  - Find the total payload mass carried by specific boosters, e.g., launched by NASA (CRS) or boosters version F9 v1.1
  - Find the dates of succesful landing outcomes.
  - Find the boosters which have success in ground pad and have specific payload mass range
  - Find the total number of successful and unsuccessful mission outcomes
  - Find the booster versions that have carried the maximum payload mass.
  - Find the month, booster version and launch_site of every succesful outcome in ground pad in a specific year, e.g., in 2017
  - Rank the count of landing outcomes in descending order during a specific time range
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Exploratory%20Data%20Analytics%20SQL%20(sqllite).ipynb

Build an Interactive Map with Folium

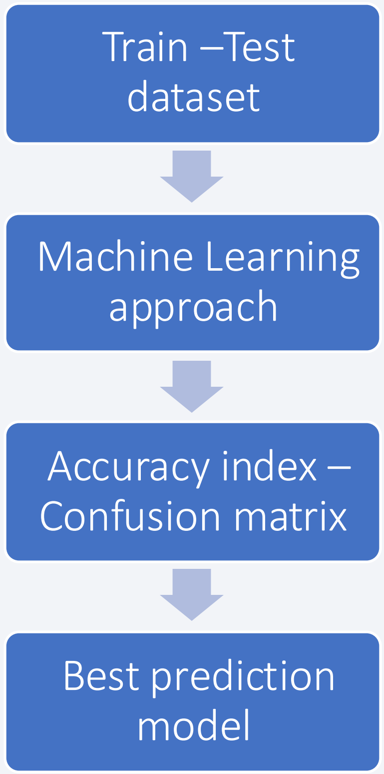
- The interactive `folium.Map` allowed us to add
  - `folium.Circle` objects to highlight specific areas, e.g. launch sites
  - `folium.Marker` objects to label each area
  - `folium.icon` to point towards the specific position of each area
  - `MarkerCluster` objects to group successful and unsuccessful landing outcomes
  - `MousePosition` object to display the exact coordinates of the cursor
  - `folium.PolyLine` objects to find distances
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Interactive%20Visual%20Analytics%20with%20Folium.ipynb

Build a Dashboard with Plotly Dash

- The `plotly-dash` dashboard allowed us to add:
  - A `dcc.Dropdown` list to select launch sites
  - A `px.pie` chart to show the total successful launches count for all sites
  - A `dcc.RangeSlider` to select payload range
  - A `px.scatter` chart to show the correlation between payload and launch success
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Interactive%20Dashboard%20with%20Plotly%20Dash/spacex-dash-app.py

Predictive Analysis (Classification)

- We used the data to build predictive models and evaluate them. To do that:
  - We set the **X input** data and the **Y output** "Class" data
  - We **preprocessed** the input data to standarize them
  - We **split** the X-Y dataset to **train-test** datasets
  - We executed Machine Learning model approaches: **Logistic Regression**, **Support Vector Machnine**, **Decision Tree Classifier**, **K Nearest Neighbors**
  - We evaluated each approach by computing their **score** and their **confusion matrices**
  - We ranked the scores to find the best model approach for our dataset
- Whole process: https://github.com/stavroulak/ML-capstone/blob/main/Machine%20Learning%20Prediction.ipynb



Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

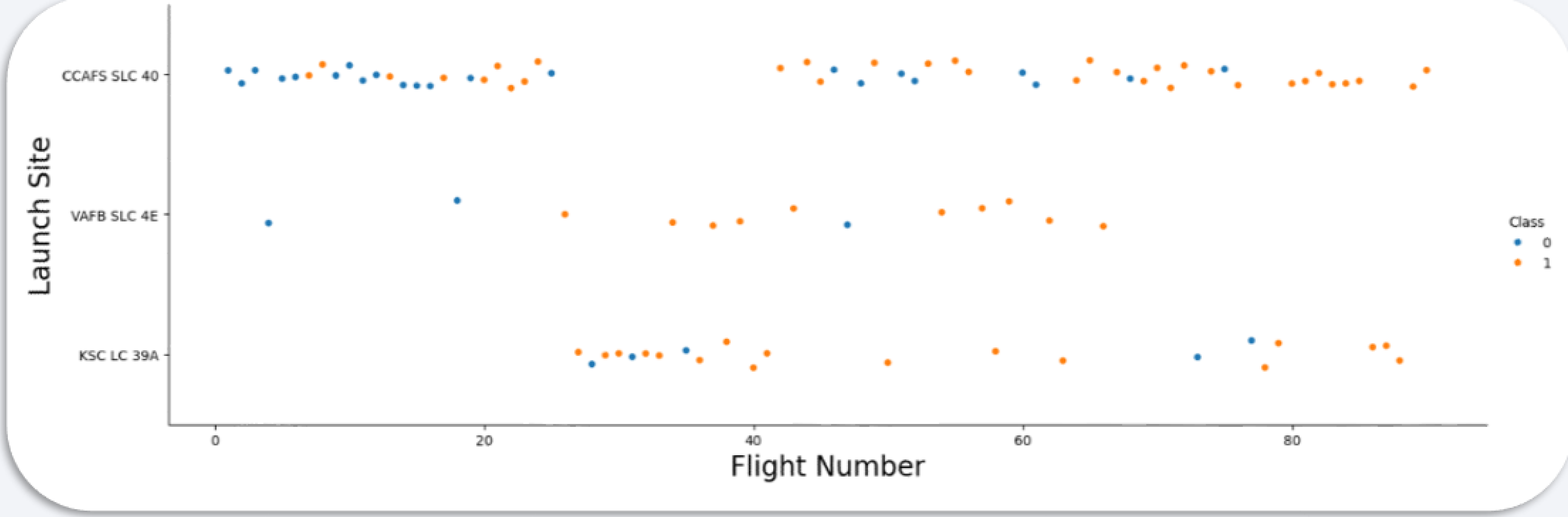
Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

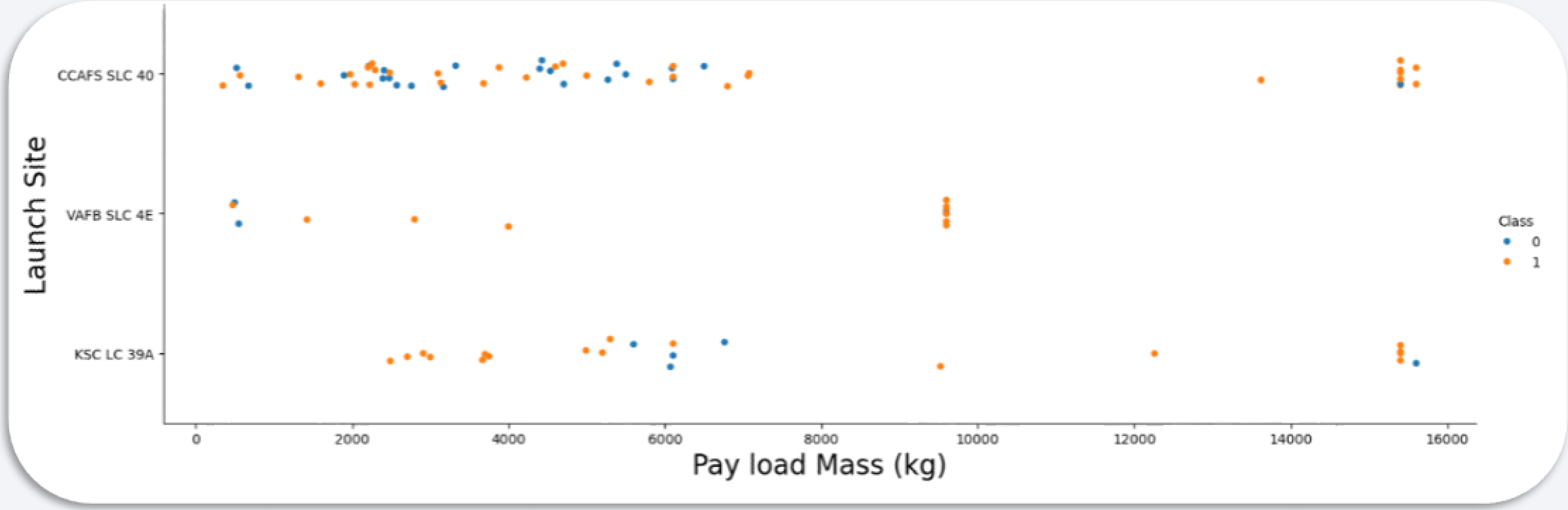
The following plot shows that:

- Most flights have been performed on the "CCAFS SLC 40" launch site. The same launch site is also described by the most extended range of Flight Number (number of continuous launch attempts),
- The relative success rate is highest on the "VAFB SLC 4E" launch site.
- In general, high number of launch attempts (Flight Number) leads to most successful outcomes (Class = 1).



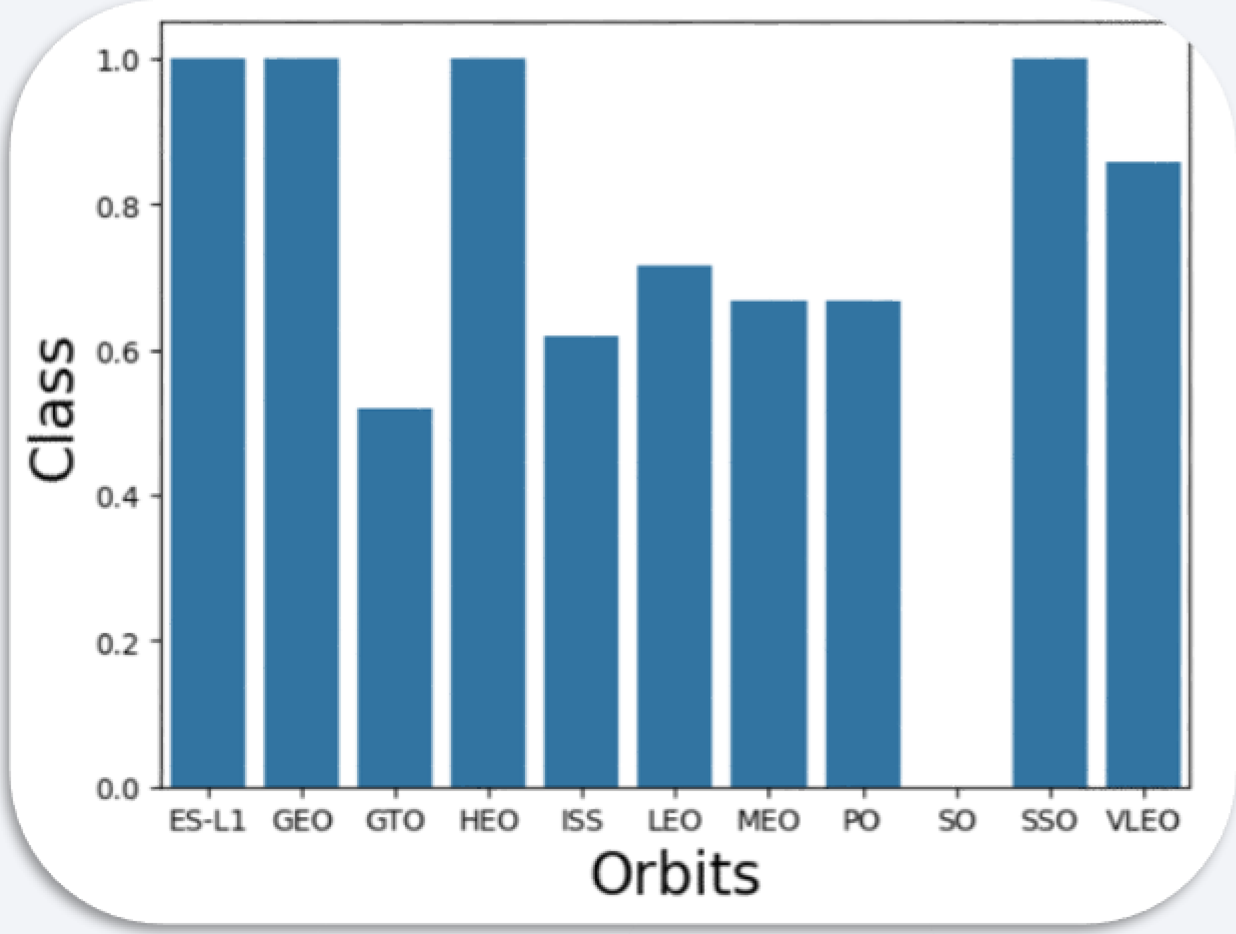
Payload vs. Launch Site

- The following plot shows that:
  - Launches on the "VAFB SLC 4E" launch site carried less payload.
  - In general, higher payload mass was leading to most successful outcomes (Class = 1).



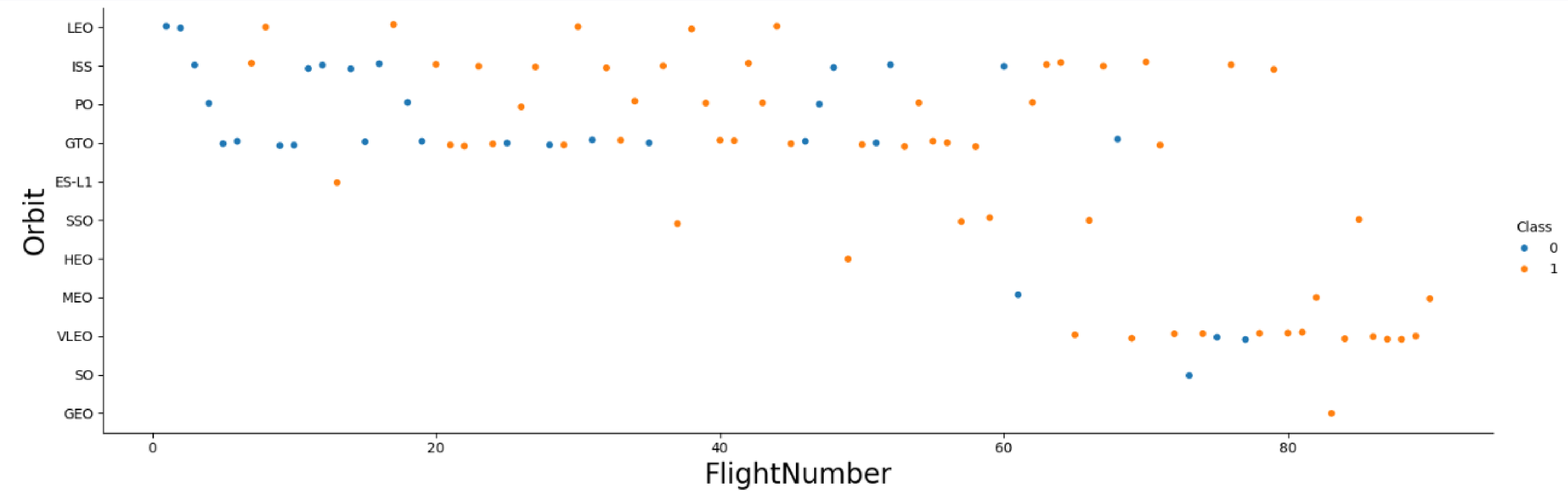
Success Rate vs. Orbit Type

- The next plot shows that:
  - The **most successful orbits are ES-L1, GEO, HEO, SSO** with 100% success rate.
  - The VLO orbit follows with a high rate of 85%.
  - ISS, LEO, MEO, PO have also a rate above 50%.
  - GTO is described by a 50% success rate.
  - The **less successful orbit is SO** with 0% success rate.



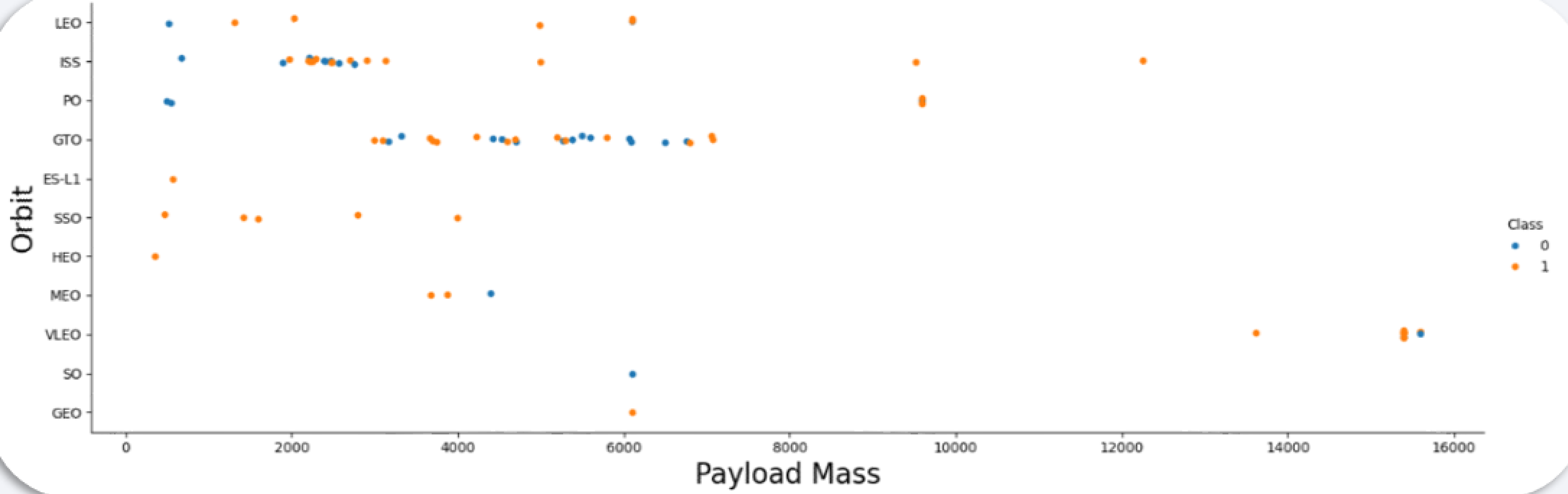
Flight Number vs. Orbit Type

- The following plot shows that:
  - In general, higher Flight Number (number of continuous attempts) leads to higher success rate.
  - Most launches have been performed in the LED, ISS, GTO and VLEO orbits.
  - The VLEO orbit has accepted the launches with the highest overall Flight Number.



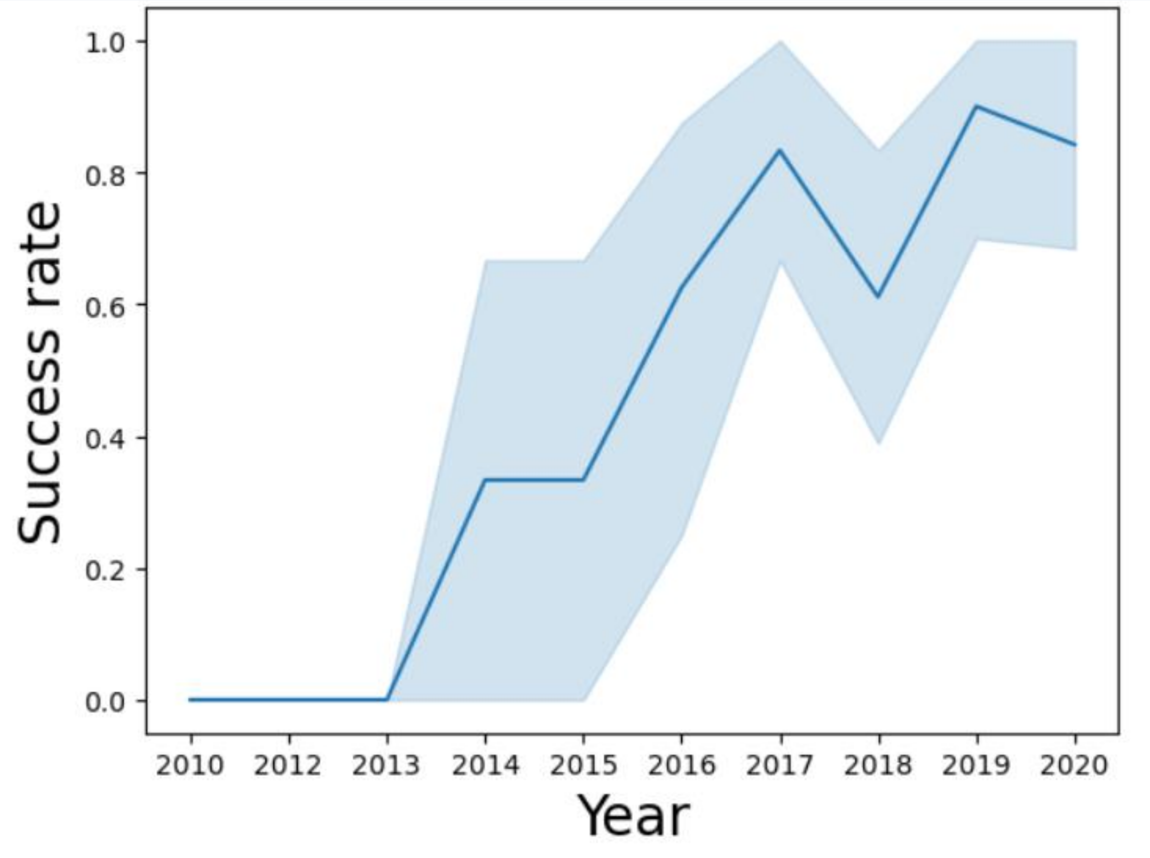
Payload vs. Orbit Type

- The following plot shows that:
  - Most launches have been carried out with 3000-7000kg payload mass in the GTO orbit and with 2000-3000kg in the ISS orbit.
  - Highest payload were carried out in the VLEO, PO and ISS launches.
  - No significant trend between payload mass and success rate has been found.



Launch Success Yearly Trend

- The next plot shows that:
  - The success rate before 2013 was 0%.
  - Since 2013, the success rate is increasing except for a drop around 2018.
  - The most rapid positive trend occurred between 2013 and 2017, where the success rate increased from 0% to 70-90%.



All Launch Site Names

- Using the `SPACEXTBL` table and the corresponding SQL query we displayed the distinct values of the `Launch_Site` column

```
sql_query =
'SELECT DISTINCT "Launch_Site"
 as "Launch_Sites"
 FROM SPACEXTBL'
```

	Launch_Sites
0	CCAFS LC-40
1	VAFB SLC-4E
2	KSC LC-39A
3	CCAFS SLC-40

Launch Site Names Begin with 'KSC'

- Using the `SPACEXTBL` table and the corresponding SQL query we displayed five table entries with `Launch_Site` values starting with specific letters

```
sql_query = 'SELECT * FROM SPACEXTBL
             WHERE LAUNCH_SITE LIKE "KSC%" LIMIT 5'
```

	Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
0	2017-02-19	14:39:00	F9 FT B1031.1	KSC LC-39A	SpaceX CRS-10	2490	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
1	2017-03-16	6:00:00	F9 FT B1030	KSC LC-39A	EchoStar 23	5600	GTO	EchoStar	Success	No attempt
2	2017-03-30	22:27:00	F9 FT B1021.2	KSC LC-39A	SES-10	5300	GTO	SES	Success	Success (drone ship)
3	2017-05-01	11:15:00	F9 FT B1032.1	KSC LC-39A	NROL-76	5300	LEO	NRO	Success	Success (ground pad)
4	2017-05-15	23:21:00	F9 FT B1034	KSC LC-39A	Inmarsat-5 F4	6070	GTO	Inmarsat	Success	No attempt

Total Payload Mass

- Using the `SPACEXTBL` table and the corresponding SQL query we computed the sum of the `PAYLOAD_MASS__KG_` column to find the total payload mass for NASA (CRS).

```
sql_query = """

            SELECT SUM(PAYLOAD_MASS__KG_)
            AS "Total PayloadMass for NASA(CRS)"

            FROM SPACEXTBL

            WHERE Customer = "NASA (CRS)"

            """
```

	Total PayloadMass for NASA(CRS)
0	45596

Average Payload Mass by F9 v1.1

- Using the `SPACEXTBL` table and the corresponding SQL query we computed the average of the `PAYLOAD_MASS__KG_` column for the `"F9 v1.1"`.

```
sql_query = """
            SELECT AVG(PAYLOAD_MASS__KG_)
            AS "Total PayloadMass for F9 v1.1"

            FROM SPACEXTBL

            WHERE Booster_Version = "F9 v1.1"
            """
```

	Total PayloadMass for F9 v1.1
0	2928.4

First Successful Ground Landing Date

- Using the SPACEXTBL table and the corresponding SQL query we found the dates of successful outcomes in drone ship. They are found as the Date entries with "Success (drone ship)" values in the Landing_Outcome column.

```
sql_query = """
            SELECT Date FROM SPACEXTBL

            WHERE Landing_Outcome
           LIKE "Success (drone ship)"
            """
```

	Date
0	2016-04-08
1	2016-05-06
2	2016-05-27
3	2016-08-14
4	2017-01-14
5	2017-03-30
6	2017-06-23
7	2017-06-25
8	2017-08-24
9	2017-10-09
10	2017-10-11
11	2017-10-30
12	2018-04-18
13	2018-05-11

Successful Drone Ship Landing with Payload between 4000 and 6000

- Using the `SPACEXTBL` table and the corresponding SQL query we found the **names of boosters** which have **successfully** landed on **ground pad** and had **payload** mass **greater than 4000 but less than 6000**.

```
sql_query = """
            SELECT Booster_Version FROM SPACEXTBL

            WHERE (Landing_Outcome
            LIKE "Success (ground pad)")

            AND (PAYLOAD_MASS__KG_ > 4000)
            AND (PAYLOAD_MASS__KG_ < 6000)
            """
```

	Booster_Version
0	F9 FT B1032.1
1	F9 B4 B1040.1
2	F9 B4 B1043.1

Total Number of Successful and Failure Mission Outcomes

- Using the `SPACEXTBL` table and the corresponding SQL query we computed the counts of `"Success"` and `"Failure"` for all the `Mission_Outcome` entries.

```
sql_query = """
            SELECT

            COUNT(Mission_Outcome LIKE "Success")

            AS "Success Count",

            COUNT(Mission_Outcome LIKE "Failure")

            AS "Failure Count"

            FROM SPAC
            """
```

	Success Count	Failure Count
0	101	101

Boosters Carried Maximum Payload

- Using the `SPACEXTBL` table and the corresponding SQL query we found the **boosters** which have carried the **maximum payload mass**.

```
sql_query = """
            SELECT Booster_Version
             FROM SPACEXTBL
             WHERE PAYLOAD_MASS__KG_ = (
               SELECT MAX(PAYLOAD_MASS__KG_)
               FROM SPACEXTBL
             )
            """
```

	Booster_Version
0	F9 B5 B1048.4
1	F9 B5 B1049.4
2	F9 B5 B1051.3
3	F9 B5 B1056.4
4	F9 B5 B1048.5
5	F9 B5 B1051.4
6	F9 B5 B1049.5
7	F9 B5 B1060.2
8	F9 B5 B1058.3
9	F9 B5 B1051.6
10	F9 B5 B1060.3
11	F9 B5 B1049.7

2015 Launch Records

- Using the `SPACEXTBL` table and the corresponding SQL query we found the **month names, booster versions, launch_sites** for the **succesful landing_outcomes in ground pad** for the months in year **2017**.

```
sql_query = """
            SELECT substr(Date,6,2)
            AS "Month",
            Booster_Version,
            Launch_Site

            FROM SPACEXTBL

            WHERE (
            substr(Date,0,5) = '2017'
            ) AND (
            Landing_Outcome
            LIKE "Success (ground pad)"
            )
            """
```

	Month	Booster_Version	Launch_Site
0	02	F9 FT B1031.1	KSC LC-39A
1	05	F9 FT B1032.1	KSC LC-39A
2	06	F9 FT B1035.1	KSC LC-39A
3	08	F9 B4 B1039.1	KSC LC-39A
4	09	F9 B4 B1040.1	KSC LC-39A
5	12	F9 FT B1035.2	CCAFS SLC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Using the SPACEXTBL table and the corresponding SQL query we ranked the count of landing outcomes between the date "2010-06-04" and "2017-03-20", in descending order.

```
sql_query = """
             SELECT Landing_Outcome,
             COUNT(Landing_Outcome)
            FROM SPACEXTBL
            WHERE (
             DATE > "2010-06-04"
             ) AND (
            DATE < "2017-03-20"
            )
            GROUP BY Landing_Outcome
            ORDER BY
             COUNT(Landing_Outcome)
             DESC
```

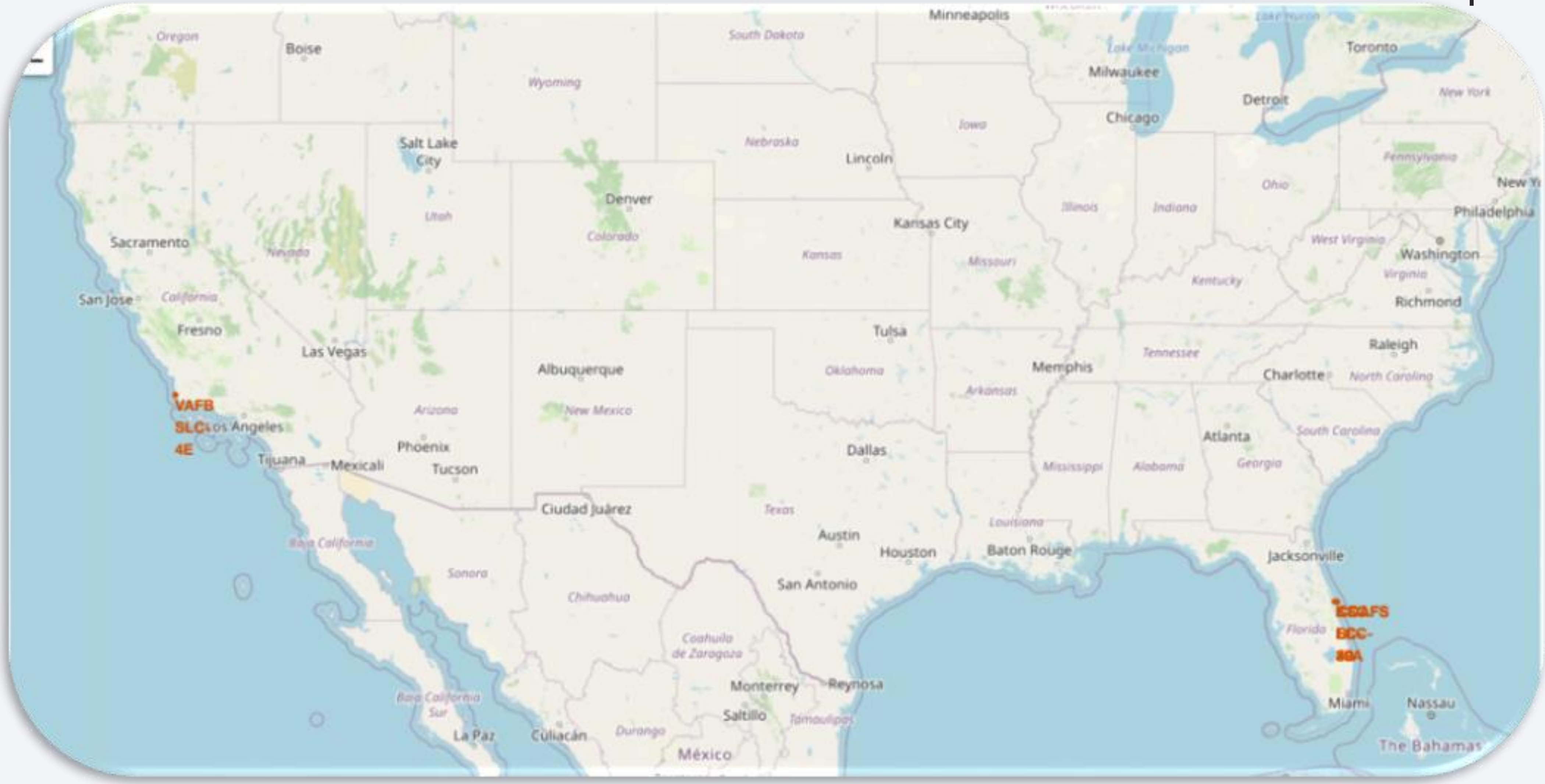
	Landing_Outcome	COUNT(Landing_Outcome)
0	No attempt	10
1	Success (drone ship)	5
2	Failure (drone ship)	5
3	Success (ground pad)	3
4	Controlled (ocean)	3
5	Uncontrolled (ocean)	2
6	Precluded (drone ship)	1
7	Failure (parachute)	1

Section 3

Launch Sites Proximities Analysis

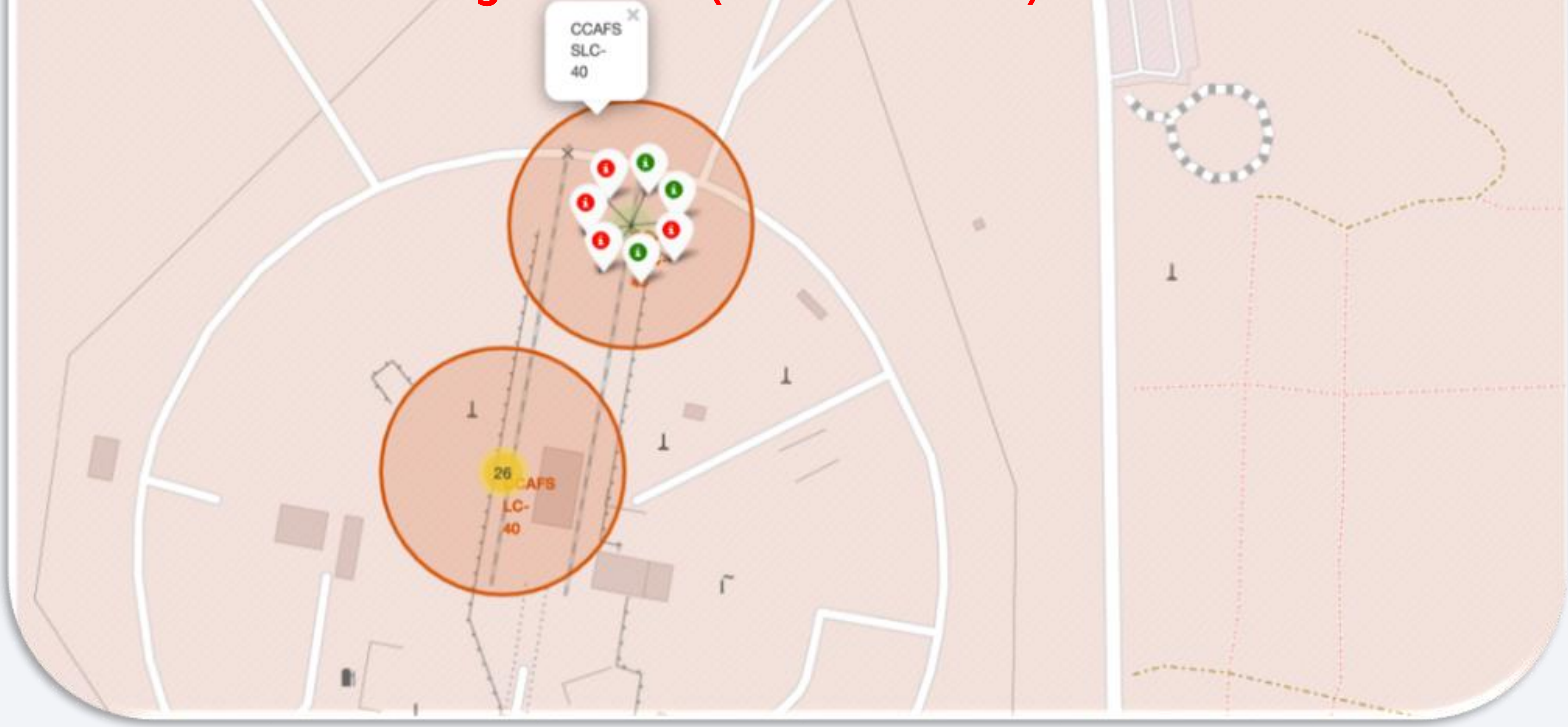
Launch sites location

- The annotated launch sites are located close to the coast and close to the Equator.

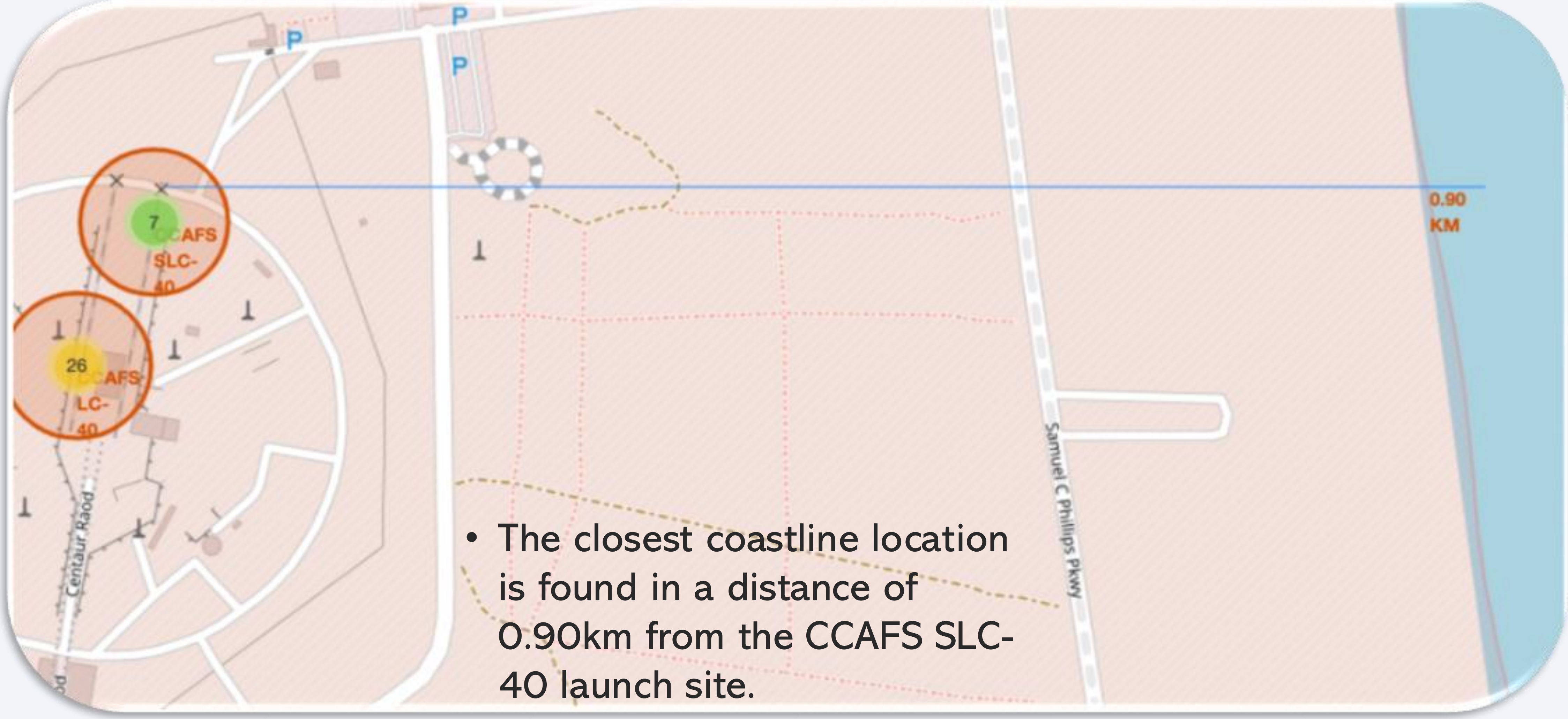


Clustered launch sites and colored annotations

- Clustered launch sites location (red large circle) could zoom-in to specific launch sites with **successful landing outcome (green annotations)** and **unsuccessful landing outcome (red annotation)**.



Launch Sites distance to close landmark



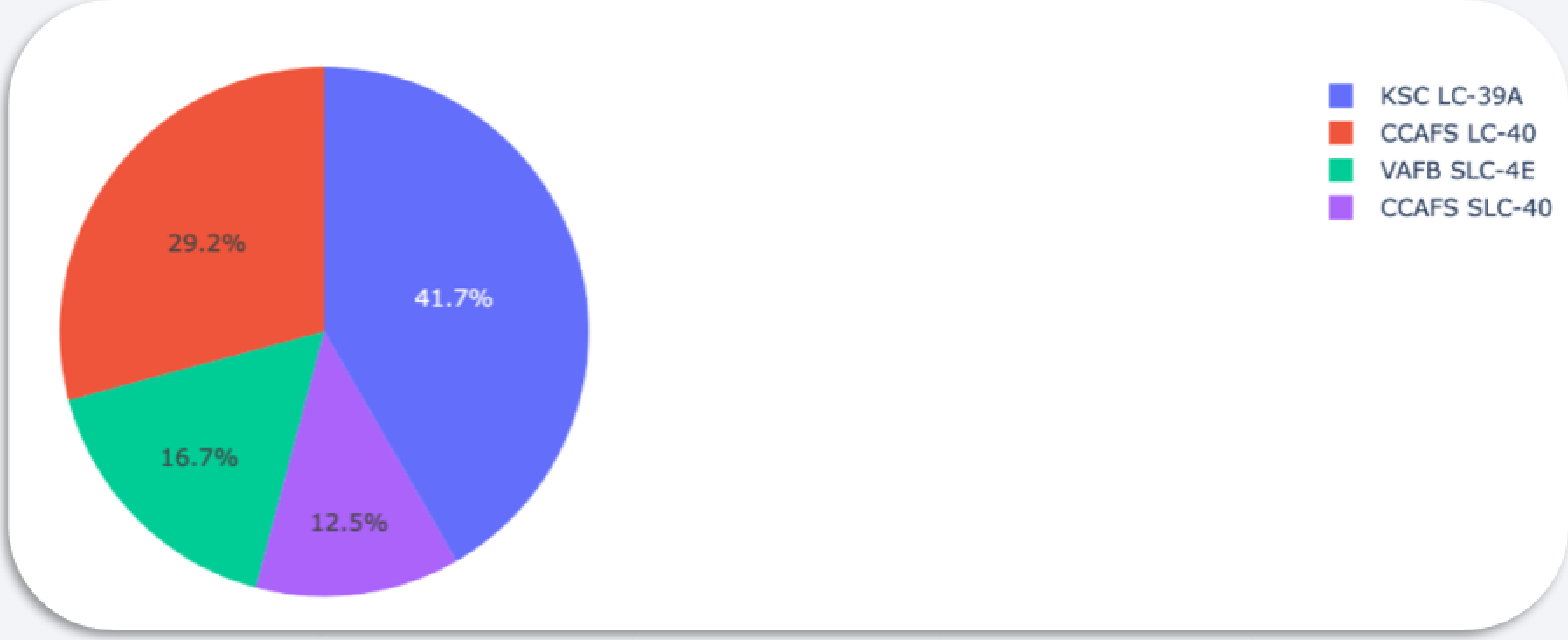
- The closest coastline location is found in a distance of 0.90km from the CCAFS SLC-40 launch site.

Section 4

Build a Dashboard with Plotly Dash

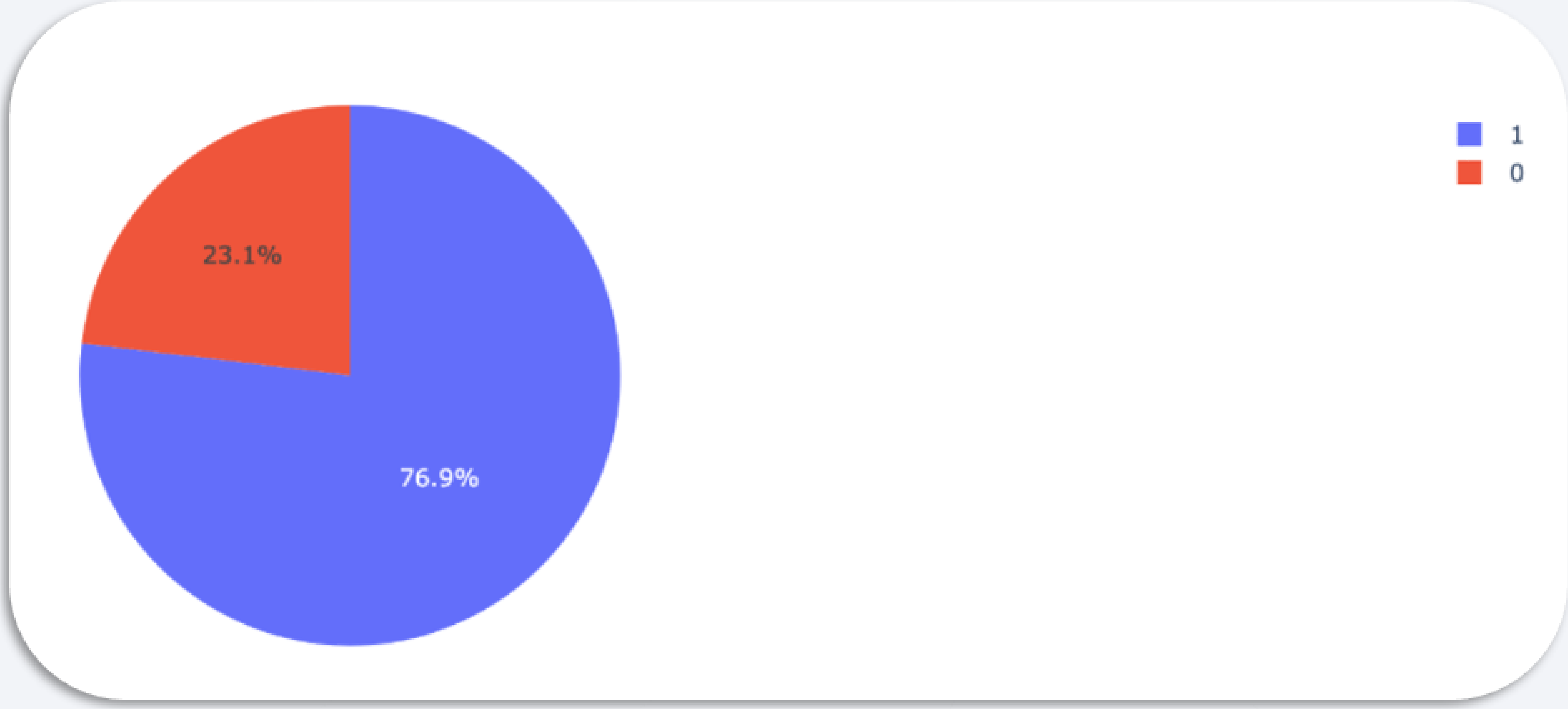
Total launch success rate for each site

- **KSC LC-39A** has the **highest success rate (41.7%)** relative to the total launch success rate. CCAFS LC-40 follows with 29.2%.



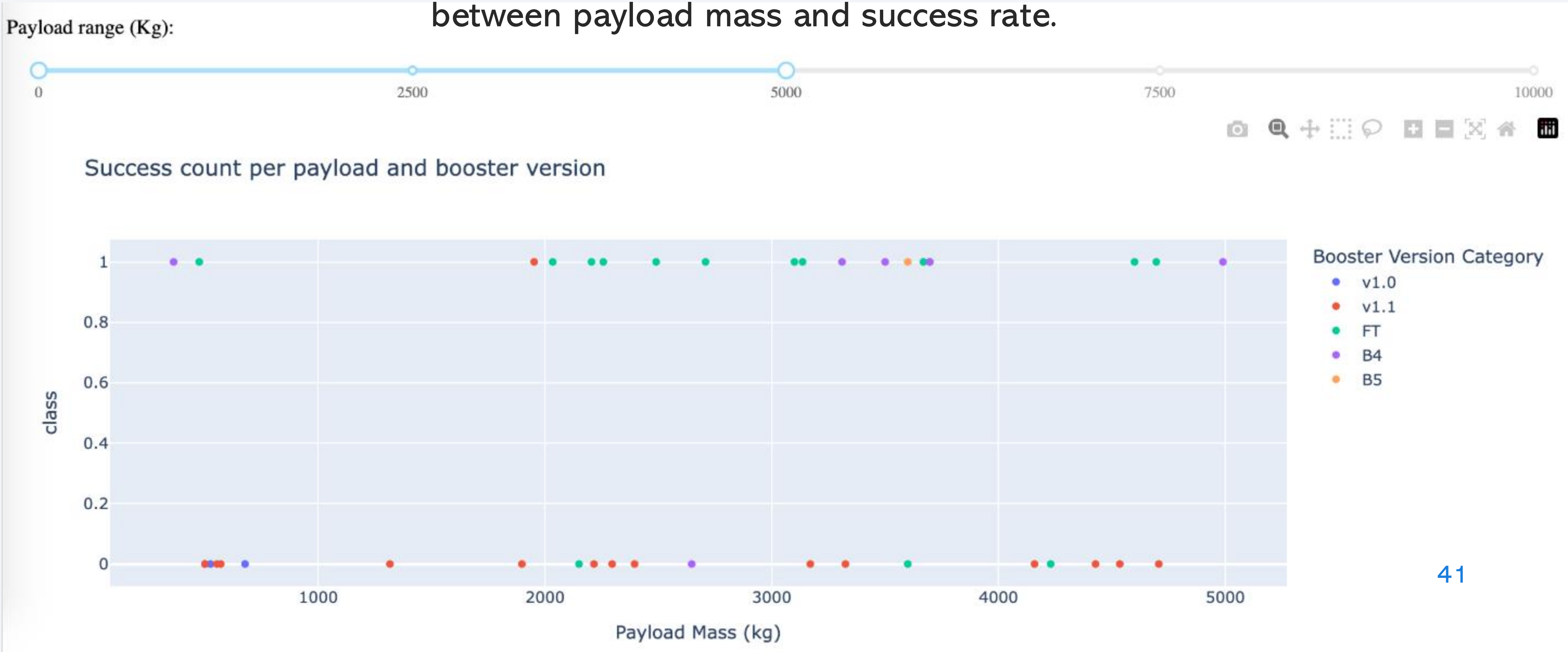
Success ratio of the most successful launch site

- A fraction of **76.9% of the total launches from KSC LC-39A**, the launch site ranked first in successful landing outcomes, **were successful**.



Success count for payload up to 5000 kg

- At low payload mass ranges, there is no correlation between payload mass and success rate.



Section 5

Predictive Analysis (Classification)

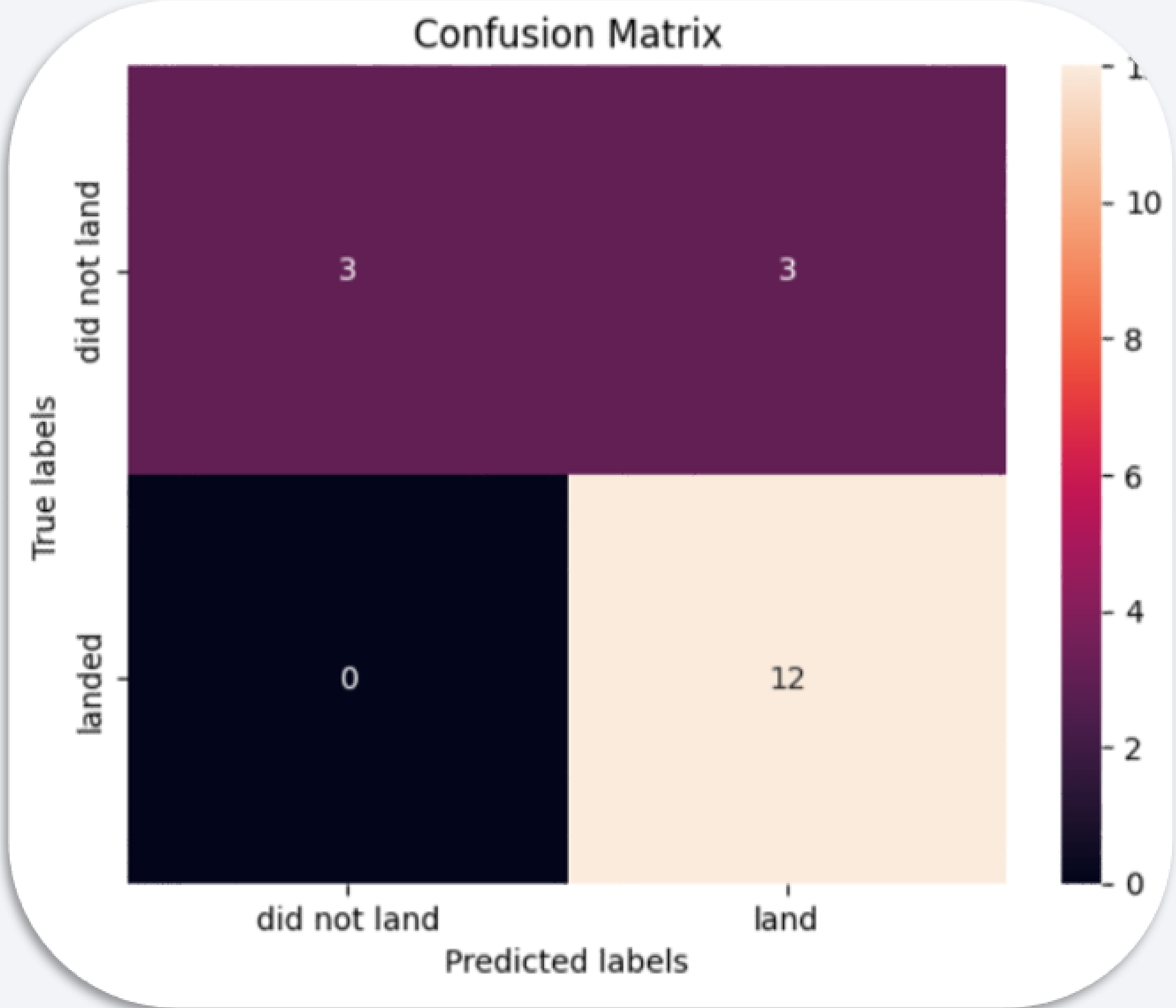
Classification Accuracy

- By ranking each built classification model, we found that the **DecisionTree** has the highest classification accuracy for our dataset.

```
In [44]:  scores = [
              logreg_cv.best_score_,
              svm_cv.best_score_,
              tree_cv.best_score_,
              knn_cv.best_score_
          ]
          models = [
              'LogisticRegression',
              'SupportVector',
              'DecisionTree',
              'KNeighbors',
          ]

          models_df = pd.DataFrame({
              "Model": models,
              "Accuracy": scores
          })

          models_df = models_df.sort_values(by="Accuracy", ascending=False)

          print("Best model: ", models_df.iloc[0,0])
          print("with score:", models_df.iloc[0,1])

         Best model:  DecisionTree
         with score: 0.8875
```

Confusion Matrix

- The confusion matrix of the Decision Tree highlights the model performance in the test dataset:
  - The build model managed to predict all the 12 "landed" cases successfully.
  - The model managed to predict the half "did not land" cases: 3 out of 6.



Conclusions

- Higher flight number leads to higher success rates.
- Launch success rate is increasing since 2013.
- KSC LC-39A have the most relative success launch rate of any sites (41.7%).
- ES-L1, GEO, HEO, SSO orbits have a 100% success rate.
- The Tree Classifier Algorithm is the best Machine Learning approach for this dataset with an accuracy score of 88.75%.

Thank you!